AI Lab assignment 9

Tejas Patil
U24AI061

Q1)

```cpp
#include <bits/stdc++.h>
using namespace std;

#define HUMAN 'O'
#define AI 'X'
#define EMPTY '_'

int nodes = 0;

// Print board
void printBoard(vector<vector<char>> &board) {
    for (auto &row : board) {
        for (auto &cell : row) cout << cell << " ";
        cout << endl;
    }
    cout << "-----------\n";
}

// Evaluate board
int evaluate(vector<vector<char>> &b) {
    for (int i = 0; i < 3; i++) {
        if (b[i][0] == b[i][1] && b[i][1] == b[i][2]) {
            if (b[i][0] == AI) return 10;
            if (b[i][0] == HUMAN) return -10;
        }
        if (b[0][i] == b[1][i] && b[1][i] == b[2][i]) {
            if (b[0][i] == AI) return 10;
            if (b[0][i] == HUMAN) return -10;
        }
    }

    if (b[0][0] == b[1][1] && b[1][1] == b[2][2]) {
        if (b[0][0] == AI) return 10;
```

```
        if (b[0][0] == HUMAN) return -10;
    }

    if (b[0][2] == b[1][1] && b[1][1] == b[2][0]) {
        if (b[0][2] == AI) return 10;
        if (b[0][2] == HUMAN) return -10;
    }

    return 0;
}

bool isMovesLeft(vector<vector<char>> &board) {
    for (auto &row : board)
        for (auto &cell : row)
            if (cell == EMPTY) return true;
    return false;
}

// Minimax with tree printing
int minimax(vector<vector<char>> &board, bool isMax, int depth) {
    nodes++;

    int score = evaluate(board);

    // Print node (visualization)
    cout << string(depth * 2, ' ') << "Depth " << depth
         << " | Score: " << score << endl;

    if (score == 10 || score == -10) return score;
    if (!isMovesLeft(board)) return 0;

    if (isMax) {
        int best = -1000;

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] == EMPTY) {
                    board[i][j] = AI;
                    best = max(best, minimax(board, false, depth + 1));
                    board[i][j] = EMPTY;
```

```
                }
            }
        }
        return best;
    } else {
        int best = 1000;

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] == EMPTY) {
                    board[i][j] = HUMAN;
                    best = min(best, minimax(board, true, depth + 1));
                    board[i][j] = EMPTY;
                }
            }
        }
        return best;
    }
}

// Find best move
pair<int,int> findBestMove(vector<vector<char>> &board) {
    int bestVal = -1000;
    pair<int,int> bestMove = {-1, -1};

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == EMPTY) {
                board[i][j] = AI;

                int moveVal = minimax(board, false, 0);

                board[i][j] = EMPTY;

                if (moveVal > bestVal) {
                    bestMove = {i, j};
                    bestVal = moveVal;
                }
            }
        }
```

```
    }
    return bestMove;
}

int main() {
    vector<vector<char>> board = {
        { 'X', 'O', 'X' },
        { 'O', 'O', '_' },
        { '_', '_', 'X' }
    };

    cout << "Initial Board:\n";
    printBoard(board);

    nodes = 0;
    pair<int,int> bestMove = findBestMove(board);

    cout << "\nBest Move (Minimax): ("
         << bestMove.first << ", " << bestMove.second << ")\n";

    cout << "Total Nodes Explored: " << nodes << endl;

    return 0;
}
```

```
Initial Board:
X O X
O O _
_ _ X
-----------
Depth 0 | Score: 10
Depth 0 | Score: 0
  Depth 1 | Score: -10
  Depth 1 | Score: -10
Depth 0 | Score: 0
  Depth 1 | Score: -10
  Depth 1 | Score: 0
    Depth 2 | Score: 10

Best Move (Minimax): (1, 2)
Total Nodes Explored: 8
```

Q2)

```cpp
#include<bits/stdc++.h>
using namespace std;

const int INF = 1e9;
int n = 14;

vector<string> city = {
    "Chicago","Indianapolis","Columbus","Cleveland","Detroit",
    "Buffalo","Pittsburgh","Baltimore","Philadelphia",
    "New York","Boston","Providence","Syracuse","Portland"
};

vector<vector<pair<int,int>>> adj(n);

// Alpha-Beta function
int alphaBeta(int curr, int dest, int depth, bool isMax,
              int alpha, int beta, vector<bool>& visited) {

    // If reached destination → leaf node
    if(curr == dest) {
        cout << "Depth " << depth << " | Reached " << city[curr]
             << " → Return 0\n";
        return 0;
    }

    visited[curr] = true;

    if(isMax) {
        int best = -INF;

        cout << "\n[ MAX Node ] City: " << city[curr]
             << " Depth: " << depth
             << " Alpha: " << alpha << " Beta: " << beta << "\n";

        for(auto &p : adj[curr]) {
            int next = p.first;
            int wt = p.second;
```

```
        if(!visited[next]) {
            int val = wt + alphaBeta(next, dest, depth+1, false,
                                     alpha, beta, visited);

            best = max(best, val);
            alpha = max(alpha, best);

            cout << "  -> Visiting " << city[next]
                 << " | Cost: " << val
                 << " | Updated Alpha: " << alpha << "\n";

            if(beta <= alpha) {
                cout << "   PRUNED at " << city[next]
                     << " (Beta <= Alpha)\n";
                break;
            }
        }
    }

    visited[curr] = false;
    return best;
}
else {
    int best = INF;

    cout << "\n[ MIN Node ] City: " << city[curr]
         << " Depth: " << depth
         << " Alpha: " << alpha << " Beta: " << beta << "\n";

    for(auto &p : adj[curr]) {
        int next = p.first;
        int wt = p.second;

        if(!visited[next]) {
            int val = wt + alphaBeta(next, dest, depth+1, true,
                                     alpha, beta, visited);

            best = min(best, val);
            beta = min(beta, best);
```

```
                cout << "  -> Visiting " << city[next]
                     << " | Cost: " << val
                     << " | Updated Beta: " << beta << "\n";

                if(beta <= alpha) {
                    cout << "  PRUNED at " << city[next]
                         << " (Beta <= Alpha)\n";
                    break;
                }
            }
        }

        visited[curr] = false;
        return best;
    }
}

int main() {

    vector<vector<int>> dist(n, vector<int>(n, INF));

    for(int i = 0; i < n; i++)
        dist[i][i] = 0;

    dist[0][4] = dist[4][0] = 283;
    dist[0][3] = dist[3][0] = 345;
    dist[0][1] = dist[1][0] = 182;
    dist[1][2] = dist[2][1] = 176;
    dist[2][3] = dist[3][2] = 144;
    dist[2][6] = dist[6][2] = 185;
    dist[3][4] = dist[4][3] = 169;
    dist[3][6] = dist[6][3] = 134;
    dist[3][5] = dist[5][3] = 189;
    dist[4][5] = dist[5][4] = 256;
    dist[5][12] = dist[12][5] = 150;
    dist[5][6] = dist[6][5] = 215;
    dist[6][8] = dist[8][6] = 305;
    dist[6][7] = dist[7][6] = 247;
    dist[7][8] = dist[8][7] = 101;
```

```
    dist[8][9] = dist[9][8] = 97;
    dist[8][12] = dist[12][8] = 253;
    dist[9][10] = dist[10][9] = 215;
    dist[9][11] = dist[11][9] = 181;
    dist[10][11] = dist[11][10] = 50;
    dist[10][13] = dist[13][10] = 107;
    dist[12][10] = dist[10][12] = 312;

    // Build adjacency list
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(dist[i][j] != INF && i != j) {
                adj[i].push_back({j, dist[i][j]});
            }
        }
    }

    int src = 12; // Syracuse
    int dest = 0; // Chicago

    vector<bool> visited(n, false);

    cout << "=== Alpha-Beta Pruning Simulation ===\n";

    int result = alphaBeta(src, dest, 0, true, -INF, INF, visited);

    cout << "\n\nFinal Result (Cost) = " << result << endl;

    return 0;
}
```